Deep Learning-Based Anomaly Detection in Network Traffic: A Comparative Study

Ahmad Fauzi[1], Siti Rahayu[2], Budi Santoso[3]
[1]*Dept. of Computer Science*, *Universitas Indonesia*, Jakarta, Indonesia
[2]*Dept. of Electrical Engineering*, *Institut Teknologi Bandung*, Bandung, Indonesia
[3]*Dept. of Informatics*, *Universitas Gadjah Mada*, Yogyakarta, Indonesia
[1]ahmad.fauzi@ui.ac.id [2]s.rahayu@itb.ac.id [3]b.santoso@ugm.ac.id

***Abstract*—Network intrusion detection remains a critical challenge in modern cybersecurity. Traditional signature-based systems fail to detect novel or zero-day attacks, motivating the use of machine learning approaches. In this paper, we present a comparative study of three deep learning architectures — Long Short-Term Memory (LSTM), Convolutional Neural Network (CNN), and Transformer-based models — applied to the problem of anomaly detection in network traffic. We evaluate all models on the CICIDS-2017 and NSL-KDD benchmark datasets. Our proposed Transformer-based model achieves a detection accuracy of 98.7% with a false positive rate of 0.8%, outperforming existing state-of-the-art methods. We further analyze computational trade-offs relevant to deployment in resource-constrained edge environments.**

***Index Terms*—anomaly detection, deep learning, intrusion detection system, LSTM, Transformer, network security, edge computing**

I. Introduction

The rapid growth of internet-connected devices and services has dramatically expanded the attack surface available to malicious actors [1]. Intrusion Detection Systems (IDS) serve as a key defensive layer, monitoring network flows for suspicious behaviour. Classical IDS rely on hand-crafted signatures and rules that require continuous manual maintenance and are inherently blind to previously unseen attack vectors [2].

Machine learning has emerged as a compelling alternative, enabling systems to learn the statistical fingerprint of benign traffic and flag deviations without requiring explicit rule authoring. Among modern architectures, deep neural networks have delivered the strongest empirical results on public benchmarks [3].

A. Motivation

Despite impressive results in controlled evaluations, deploying deep learning IDS in practice raises several open questions:

- How do recurrent, convolutional, and attention-based architectures compare under identical conditions?
- What is the accuracy–latency trade-off for each family?
- Are these models robust to class imbalance, which is endemic in real network datasets?

B. Contributions

This paper makes the following contributions:

1) A rigorous head-to-head comparison of LSTM, CNN, and Transformer models on two widely-used benchmark datasets.
2) A lightweight Transformer variant (*NetFormer*) designed for edge deployment with $<$500k parameters.
3) An open-source reproduction package including preprocessing scripts, model definitions, and training notebooks.

The remainder of this paper is organised as follows. Section II reviews related work. Section III describes our methodology. Section IV presents experimental results. Section V discusses findings and limitations. Section VI concludes the paper.

II. Related Work

A. Traditional Approaches

Early IDS such as Snort [4] rely on pattern matching against known attack signatures. While highly precise for known threats, they exhibit zero generalisation to novel exploits. Statistical approaches, including k-Nearest Neighbour and Support Vector Machines, improved generalisation but struggle with high-dimensional, high-rate packet streams [5].

B. Deep Learning for Network Security

Recurrent architectures, particularly LSTMs, were among the first deep models applied to IDS due to their natural handling of sequential data. Kim *et al.* [6] demonstrated that a two-layer LSTM achieves 97.1% accuracy on NSL-KDD, outperforming shallow classifiers by a significant margin.

CNNs were later adapted for traffic classification by treating fixed- length flow windows as one-dimensional signals [7]. Their parallelism advantage over recurrent networks makes them attractive for high-throughput inference.

Attention mechanisms and Transformer architectures, originally developed for natural language processing [8], have recently been applied to time-series anomaly detection [9]. However, their applicability to low-latency network monitoring remains under-explored.

III. Methodology

A. Datasets

We evaluate on two standard benchmarks:

NSL-KDD [10]: A refined version of the KDD Cup 1999 dataset containing 125,973 training and 22,544 test records across four attack categories: DoS, Probe, R2L, and U2R.

CICIDS-2017 [11]: A modern dataset generated in a controlled environment containing 2.8 million flow records with 14 attack types including brute-force, XSS, and infiltration attacks. Table I summarises key statistics.

TABLE I
DATASET STATISTICS

Dataset	Records	Features	Attack %
NSL-KDD (train)	125,973	41	46.5%
NSL-KDD (test)	22,544	41	46.0%
CICIDS-2017	2,830,743	78	16.8%

B. Pre-processing

All categorical features are one-hot encoded. Continuous features are normalised to zero mean and unit variance using statistics computed on the training split only to prevent data leakage. Missing values, present in 0.3% of CICIDS-2017 records, are imputed with the feature median.

Class imbalance is addressed using Synthetic Minority Over-sampling Technique (SMOTE) [12] on the training set, yielding a balanced 50/50 split of benign and attack traffic.

C. Model Architectures

1) LSTM Baseline: Our LSTM model consists of two stacked LSTM layers with hidden dimension $d = 128$, followed by dropout ($p = 0.3$) and a fully-connected classification head:

$$h_t = \text{LSTM}(x_t, h_{t-1}, c_{t-1}) \tag{1}$$

$$\hat{y} = \sigma(W_o \cdot h_T + b_o) \tag{2}$$

where h_T is the final hidden state, $W_o \in \mathbb{R}^{2\times d}$ and $b_o \in \mathbb{R}^2$ are learnable parameters.

2) CNN Baseline: The CNN model applies three parallel convolutional filters of widths $\{3, 5, 7\}$ to each input window, followed by max-over-time pooling and concatenation:

$$z_k = \max_t \left(\text{ReLU}(W_k * x + b_k)\right), \quad k \in \{1, 2, 3\} \tag{3}$$

$$\hat{y} = \text{softmax}(W_c \cdot [z_1; z_2; z_3] + b_c) \tag{4}$$

3) NetFormer (Proposed): Our proposed *NetFormer* replaces the recurrence and convolution with multi-head self-attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{5}$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W^O \tag{6}$$

We use $h = 4$ attention heads, model dimension $d_{\text{model}} = 64$, two encoder layers, and a feed-forward inner dimension of 128, yielding 488k total parameters.

D. Training Protocol

All models are implemented in PyTorch and trained for 50 epochs using the Adam optimiser with learning rate $\eta = 10^{-3}$ and weight decay $\lambda = 10^{-4}$. A cosine annealing schedule reduces η to 10^{-5} by the final epoch. Batch size is 512. Early stopping with patience 10 is applied on the validation loss.

Algorithm 1 summarises the training procedure.

Algorithm 1 Model Training Procedure

Require: Dataset $\mathcal{D}$, model f_θ, epochs E
Split $\mathcal{D}$ into train / val / test (70/15/15)
Apply SMOTE to training split
Normalise features using training statistics
for epoch $e = 1$ to E **do**
  for each mini-batch $(X, y) \in \mathcal{D}_{\text{train}}$ **do**
    $\hat{y} \leftarrow f_\theta(X)$
    $\mathcal{L} \leftarrow \text{CrossEntropy}(\hat{y}, y)$
    $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$
  end for
  Evaluate on $\mathcal{D}_{\text{val}}$; update LR scheduler
  if early stopping criterion met **then**
    break
  end if
end for
return best θ by validation loss

IV. EXPERIMENTS

A. Evaluation Metrics

We report Accuracy (ACC), Precision (PRE), Recall (REC), F_1-score, and False Positive Rate (FPR):

$$\text{ACC} = \frac{TP + TN}{TP + TN + FP + FN} \tag{7}$$

$$F_1 = \frac{2 \cdot \text{PRE} \cdot \text{REC}}{\text{PRE} + \text{REC}} \tag{8}$$

$$\text{FPR} = \frac{FP}{FP + TN} \tag{9}$$

B. Results on NSL-KDD

Table II presents binary classification results on NSL-KDD. *NetFormer* achieves the best accuracy and F_1 while maintaining the lowest false positive rate.

TABLE II
BINARY CLASSIFICATION RESULTS ON NSL-KDD

Model	ACC	PRE	REC	F1	FPR
SVM [5]	92.4	91.8	93.0	92.4	5.8
LSTM [6]	97.1	96.8	97.4	97.1	2.4
CNN	97.5	97.2	97.8	97.5	2.1
NetFormer (ours)	**98.7**	**98.5**	**98.9**	**98.7**	**0.8**

C. Results on CICIDS-2017

Table III shows multi-class results across all 14 attack categories on CICIDS-2017. NetFormer again leads on macro-averaged F_1, with particular strength on the low-frequency U2R and R2L categories that challenge recurrent baselines.

TABLE III
MULTI-CLASS RESULTS ON CICIDS-2017 (MACRO AVG.)

Model	ACC	Macro-F1	FPR
LSTM	95.3	91.2	3.6
CNN	96.1	92.7	3.1
NetFormer	**97.4**	**94.8**	**1.9**

D. Inference Latency

Table IV reports mean per-sample inference time measured on an NVIDIA RTX 3060 GPU and an ARM Cortex-A72 CPU (simulating edge deployment). NetFormer is $2.1\times$ faster than the LSTM at batch size 1 due to the parallelisable attention computation.

TABLE IV
INFERENCE LATENCY (μS / SAMPLE)

Model	Params	GPU	CPU
LSTM	1.2M	42	310
CNN	0.9M	28	195
NetFormer	**0.5M**	**20**	**148**

V. DISCUSSION

A. Why Transformers Outperform RNNs Here

Sequential models like LSTM process tokens one at a time, causing gradient attenuation over long flow windows. Self-attention computes pairwise interactions across the entire window in $O(n^2 d)$ operations, allowing it to capture long-range dependencies between early and late packets that are diagnostically significant for slow-scan attacks.

B. Limitations

Several limitations should be noted:

- Both benchmark datasets are synthetic or semi-synthetic; generalisation to live production traffic is unverified.
- We do not evaluate adversarial robustness — a motivated attacker may craft flows that evade the learned detector.
- SMOTE may introduce artefacts for highly imbalanced minority classes such as U2R.

C. Future Work

Promising directions include: federated learning across multiple network vantage points to improve generalisation without centralising sensitive traffic; contrastive pre-training on unlabelled flows; and formal verification of detection boundaries.

VI. CONCLUSION

We presented a systematic comparison of LSTM, CNN, and Transformer architectures for network anomaly detection. Our proposed *NetFormer* model achieves 98.7% accuracy on NSL-KDD and 97.4% on CICIDS-2017, with a false positive rate of 0.8% and 1.9% respectively — improving over prior state-of-the-art whilst reducing parameter count by $2.4\times$. The model's low latency makes it viable for edge deployment, and we release all code and pre-trained weights to support reproducibility.

REFERENCES

[1] Cisco Systems, "Cisco Annual Internet Report 2022–2027," *White Paper*, 2023.

[2] H.-J. Liao, C.-H. R. Lin, Y.-C. Lin, and K.-Y. Tung, "Intrusion detection system: A comprehensive review," *J. Network and Computer Applications*, vol. 36, no. 1, pp. 16–24, 2013.

[3] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, and A. Hotho, "A survey of network-based intrusion detection data sets," *Computers & Security*, vol. 86, pp. 147–166, 2019.

[4] M. Roesch, "Snort — Lightweight intrusion detection for networks," in *Proc. USENIX LISA*, 1999, pp. 229–238.

[5] C.-F. Tsai, Y.-F. Hsu, C.-Y. Lin, and W.-Y. Lin, "Intrusion detection by machine learning: A review," *Expert Systems with Applications*, vol. 36, no. 10, pp. 11994–12000, 2009.

[6] J. Kim, J. Kim, H. L. T. Thu, and H. Kim, "Long short term memory recurrent neural network classifier for intrusion detection," in *Proc. IEEE ICPADS*, 2016, pp. 1–4.

[7] W. Wang *et al.*, "HAST-IDS: Learning hierarchical spatial-temporal features using deep neural networks to improve intrusion detection," *IEEE Access*, vol. 6, pp. 1792–1806, 2017.

[8] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[9] J. Xu *et al.*, "Anomaly transformer: Time series anomaly detection with association discrepancy," in *Proc. ICLR*, 2022.

[10] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE CISDA*, 2009, pp. 1–6.

[11] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. ICISSP*, 2018, pp. 108–116.

[12] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.